

Black box testing focuses on validating software functionality without knowing its internal code. The “Ws” (What, Why, When, Where, Who) and “How” help structure the techniques, while examples and exercises reinforce practical understanding.

 The Ws of Black Box Testing

- **What:** Testing based on inputs/outputs and requirements, ignoring internal code.
 - **Why:** Ensures the system meets user expectations and requirements.
 - **When:** Applied during system testing, acceptance testing, and regression testing.
 - **Where:** Used across applications—websites, mobile apps, APIs, databases.
 - **Who:** Testers, QA engineers, or end-users validating functionality.
-

 How Black Box Testing Works

1. **Understand requirements/specifications.**
 2. **Identify inputs and expected outputs.**
 3. **Design test cases** using techniques like equivalence partitioning, boundary value analysis, decision tables, and state transition testing.
 4. **Execute tests** without looking at source code.
 5. **Compare actual vs expected results.**
 6. **Report defects** if mismatches occur.
-

 Key Techniques with Examples

Technique	How it Works	Example
Equivalence Partitioning	Divide inputs into valid/invalid classes	Input field accepts ages 18–60. Test one value from each partition: 25 (valid), 10 (invalid), 70 (invalid).
Boundary Value Analysis	Test at edges of input ranges	For ages 18–60, test 17, 18, 60, 61.

Technique	How it Works	Example
Decision Table Testing	Use rules/conditions to design tests	Online shopping: If payment successful + stock available → order confirmed. If payment fails → order rejected.
State Transition Testing	Validate system behavior across states	ATM: Insert card → enter PIN → withdraw cash. Test invalid PIN transitions.
Error Guessing	Based on tester's intuition	Enter special characters in username field to check handling.



Exercises to Validate Knowledge

1. Equivalence Partitioning Exercise

- A login form accepts passwords 8–12 characters long.
- Identify partitions and test cases.

2. Boundary Value Analysis Exercise

- A system allows up to 100 users.
- Test cases: 0, 1, 100, 101.

3. Decision Table Exercise

- Rules:
 - If user is premium + payment made → access granted.
 - If user is free + payment made → access denied.
 - If user is premium + no payment → access denied.
- Build decision table and test cases.

4. State Transition Exercise

- Elevator system: States = Idle, Moving Up, Moving Down, Door Open.
- Test transitions like “Idle → Moving Up → Door Open.”



How to Practice

- **Create test cases** for everyday apps (e.g., WhatsApp login, Amazon checkout).

- **Compare outputs** with expected behavior.
 - **Document defects** and suggest fixes.
 - **Peer review** test cases to spot gaps.
-

More Examples by Technique

1. Equivalence Partitioning

- **Scenario:** Online form accepts salary input between 10,000–100,000.
 - **Partitions:**
 - Valid: 20,000, 50,000, 90,000
 - Invalid: 5,000 (below), 150,000 (above), text input (“abc”)
-

2. Boundary Value Analysis

- **Scenario:** A discount applies for orders between 1–50 items.
 - **Test cases:**
 - Below boundary: 0
 - At boundary: 1, 50
 - Just outside: 51
-

3. Decision Table Testing

- **Scenario:** Loan approval rules:
 - If income > 50k AND credit score > 700 → Approve
 - If income ≤ 50k OR credit score ≤ 700 → Reject
- **Decision table:**

Income > 50k	Credit > 700	Outcome
Yes	Yes	Approve
Yes	No	Reject

Income > 50k	Credit > 700	Outcome
No	Yes	Reject
No	No	Reject

4. State Transition Testing

- **Scenario:** Online shopping cart states:
 - Empty → Add item → Active
 - Active → Checkout → Paid
 - Active → Remove item → Empty
 - **Tests:**
 - Add item to empty cart → Active
 - Checkout without items → Invalid transition
-

5. Error Guessing

- **Scenario:** File upload system.
 - **Guessed errors:**
 - Uploading unsupported file type (.exe)
 - Uploading extremely large file (>1GB)
 - Uploading with special characters in filename
-

Practice Exercises

1. Equivalence Partitioning Exercise

- A system accepts PIN codes of exactly 4 digits.
- Identify valid and invalid partitions.

2. Boundary Value Analysis Exercise

- A grading system accepts marks between 0–100.
- Write test cases for boundary values.

3. Decision Table Exercise

- Rules:
 - If user is admin → Full access
 - If user is guest → Read-only
 - If user is member + subscription active → Write access
 - If user is member + subscription inactive → Read-only
- Build decision table and test cases.

4. State Transition Exercise

- Traffic light system: Red → Green → Yellow → Red.
- Test valid and invalid transitions (e.g., Red → Yellow directly).

5. Error Guessing Exercise

- Login form:
 - Try empty username/password
 - Try SQL injection (' OR 1=1 --)
 - Try extremely long username (>100 chars)

Challenge Exercise

Design **black box test cases** for a **mobile banking app login**:

- Valid username/password combinations
 - Invalid credentials
 - Boundary values (password length)
 - State transitions (login → dashboard → logout)
 - Error guessing (special characters, empty fields, multiple failed attempts)
-

The Ws of White Box Testing

- **What:** Testing based on the internal structure, logic, and code of the software.

- **Why:** Ensures code correctness, optimizes performance, and validates paths, conditions, and loops.
- **When:** Applied during unit testing, integration testing, and regression testing.
- **Where:** Used in code-level testing, APIs, algorithms, and performance-critical modules.
- **Who:** Developers, QA engineers with programming knowledge.

 How White Box Testing Works

1. **Understand the code logic and structure.**
2. **Identify paths, conditions, and loops.**
3. **Design test cases** to cover statements, branches, and paths.
4. **Execute tests** using tools or manually.
5. **Measure coverage** (statement, branch, path, condition).
6. **Refactor or fix code** based on results.

 Key Techniques with Examples

Technique	How it Works	Example
Statement Coverage	Ensure every line of code executes at least once	If function has 10 lines, design tests so all 10 execute.
Branch Coverage	Test both true/false outcomes of decisions	For if (x > 0), test with x=5 (true) and x=-3 (false).
Path Coverage	Test all possible execution paths	For nested if statements, cover all combinations of outcomes.
Condition Coverage	Test each condition in a decision independently	For if (A && B), test A=true/B=false, A=false/B=true, etc.
Loop Testing	Validate loop execution (0, 1, many iterations)	For for(i=0;i<n;i++), test n=0, n=1, n=5.
Control Flow Testing	Analyze program flow graph	Ensure all nodes and edges are tested.

Technique	How it Works	Example
Data Flow Testing	Track variable definitions and usage	Test cases where variable is defined but not used, or used before definition.



Exercises to Validate Knowledge

1. Statement Coverage Exercise

- Code snippet:
 - if `x > 10`:
 - `print("Large")`
 - else:
 - `print("Small")`
 - `print("Done")`
 - Design test cases to cover all statements.
-

2. Branch Coverage Exercise

- Code snippet:
 - if `y % 2 == 0`:
 - `print("Even")`
 - else:
 - `print("Odd")`
 - Create test cases for both branches.
-

3. Path Coverage Exercise

- Code snippet:
- if `a > 0`:
- if `b > 0`:
- `print("Both positive")`
- else:

- `print("a positive, b not")`
 - `else:`
 - `print("a not positive")`
 - Identify all paths and design test cases.
-

4. Condition Coverage Exercise

- Code snippet:
 - `if (x > 5 && y < 10):`
 - `print("Valid")`
 - `else:`
 - `print("Invalid")`
 - Create test cases to evaluate each condition independently.
-

5. Loop Testing Exercise

- Code snippet:
 - `for i in range(n):`
 - `print(i)`
 - Test with `n=0`, `n=1`, `n=3`.
-

Challenge Exercise

Design **white box test cases** for a function that calculates factorial:

```
def factorial(n):
```

```
    if n < 0:
```

```
        return None
```

```
    elif n == 0:
```

```
        return 1
```

```
    else:
```

```
        result = 1
```



```
for i in range(1, n+1):
```

```
    result *= i
```

```
return result
```

- Cover **negative input**, **zero input**, **positive input**, and **loop execution paths**.

Comparison Table

Aspect	Black Box Testing	White Box Testing	Grey Box Testing
Knowledge of Code	No knowledge of internal code	Full knowledge of internal code	Partial knowledge of internal code
Focus	Functional behavior, inputs/outputs	Code structure, logic, paths	Both functionality and limited internal logic
Tester Role	QA engineers, end-users	Developers, technical testers	Testers with partial system knowledge
Techniques Used	Equivalence partitioning, boundary value analysis, decision tables, state transitions	Statement coverage, branch coverage, path coverage, loop testing	Combination of black box + selective white box techniques
Advantages	- Simulates real user experience		

- Finds missing functionality
- Independent of implementation | - Ensures code correctness
- Detects hidden errors
- Measures coverage | - Balanced approach
- Detects integration issues
- More realistic than pure white box | | **Disadvantages** | - Limited coverage of internal logic
- May miss hidden errors | - Requires programming knowledge
- Time-consuming | - Risk of bias

- May not cover all paths | | **Best Use Cases** | System testing, acceptance testing | Unit testing, integration testing | Security testing, integration testing, web applications |
-

Detailed Explanation

Black Box Testing

- **Definition:** Tests software functionality without looking at internal code.
- **Example:** Testing a login form by entering valid/invalid credentials and checking responses.
- **Goal:** Validate requirements and user expectations.

☐ White Box Testing

- **Definition:** Tests internal logic, code paths, and structures.
- **Example:** Writing unit tests to ensure every branch of an if-else statement executes.
- **Goal:** Ensure code correctness and optimize performance.

☒ Grey Box Testing

- **Definition:** Tester has partial knowledge of internal code and architecture.
 - **Example:** Testing a web application with knowledge of database schema but not full source code.
 - **Goal:** Combine functional validation with structural insights for better coverage.
-

Exercises to Validate Knowledge

1. Black Box Exercise

- Test an ATM withdrawal feature: valid PIN, invalid PIN, insufficient balance.

2. White Box Exercise

- Write test cases for a function calculating factorial, ensuring coverage of negative, zero, and positive inputs.

3. Grey Box Exercise

- Test an e-commerce checkout knowing the database schema (e.g., stock table), but not full backend code.
-

Key Takeaway

- **Black Box** = *User perspective* (functional testing).
 - **White Box** = *Developer perspective* (code-level testing).
 - **Grey Box** = *Hybrid perspective* (partial knowledge, integration focus).
-

Scenario: Banking App Login Feature

Feature Description

- Users enter username and password.
 - System validates credentials.
 - If valid → redirect to dashboard.
 - If invalid → show error message.
-

Testing Approaches

Black Box Testing

- **Tester Role:** QA engineer with no access to code.
 - **Focus:** Functional behavior.
 - **Test Cases:**
 - Valid credentials → login success.
 - Invalid credentials → error message.
 - Empty fields → validation error.
 - SQL injection attempt → error handling.
 - **Goal:** Ensure user experience and requirement compliance.
-

☐ **White Box Testing**

- **Tester Role:** Developer or technical tester.
 - **Focus:** Internal logic and code paths.
 - **Test Cases:**
 - Check if password hashing function executes.
 - Validate all branches in credential verification logic.
 - Test loop that checks login attempt count.
 - Ensure exception handling for database errors.
 - **Goal:** Ensure code correctness, coverage, and security.
-

 Grey Box Testing

- **Tester Role:** Tester with partial system knowledge (e.g., database schema).
 - **Focus:** Integration and security.
 - **Test Cases:**
 - Test login with knowledge of user table structure.
 - Validate session token generation and storage.
 - Check for timing attacks or brute-force protection.
 - Test login flow with expired credentials.
 - **Goal:** Combine functional and structural insights for deeper validation.
-

 Summary

Testing Type	Access Level	Focus	Example Test
Black Box	No code access	Functional	Valid/invalid login
White Box	Full code access	Structural	Password hashing logic
Grey Box	Partial code/database access	Hybrid	Token generation + login flow

 Different Testing Types with Ws & How

1. Black Box Testing

- **What:** Tests functionality without looking at internal code.
- **Why:** Ensures system meets user requirements.
- **When:** System testing, acceptance testing.
- **Where:** Web apps, mobile apps, APIs.
- **Who:** QA engineers, end-users.
- **How:** Inputs/outputs, techniques like equivalence partitioning, boundary value analysis.

Example: Test login form with valid/invalid credentials.

Exercise: A password must be 8–12 characters. Identify valid and invalid test cases.

2. White Box Testing

- **What:** Tests internal logic, code paths, and structures.
- **Why:** Ensures code correctness and coverage.
- **When:** Unit testing, integration testing.
- **Where:** Source code, algorithms, APIs.
- **Who:** Developers, technical testers.
- **How:** Techniques like statement coverage, branch coverage, path coverage.

Example: Test factorial function for negative, zero, and positive inputs.

Exercise: Write test cases for a function that checks if a number is prime. Cover all branches.

3. Grey Box Testing

- **What:** Hybrid approach with partial knowledge of internal code.
- **Why:** Detects integration issues and security flaws.
- **When:** Integration testing, penetration testing.
- **Where:** Web apps, client-server systems.
- **Who:** Testers with limited system knowledge.
- **How:** Combines functional tests with structural insights.

Example: Test e-commerce checkout knowing database schema but not full code.

Exercise: Test login with expired session tokens and invalid database entries.

4. Unit Testing

- **What:** Tests individual components or functions.
- **Why:** Ensures smallest units of code work correctly.
- **When:** During development.
- **Where:** Functions, classes, modules.
- **Who:** Developers.
- **How:** Automated frameworks (JUnit, NUnit, PyTest).

Example: Test `add(2,3)` returns 5.

Exercise: Write unit tests for a calculator's `divide()` function, including division by zero.

5. Integration Testing

- **What:** Tests combined modules working together.
- **Why:** Ensures interfaces and data flow are correct.
- **When:** After unit testing.
- **Where:** APIs, microservices, module interactions.
- **Who:** Developers, QA engineers.
- **How:** Top-down, bottom-up, or sandwich approach.

Example: Test payment gateway integration with shopping cart.

Exercise: Validate data flow between login module and dashboard module.

6. System Testing

- **What:** Tests the complete system as a whole.
- **Why:** Ensures end-to-end functionality.
- **When:** After integration testing.
- **Where:** Full application environment.
- **Who:** QA engineers.

- **How:** Functional, performance, security, usability tests.

Example: Test an airline booking system end-to-end.

Exercise: Write test cases for booking a flight with valid and invalid inputs.

7. Acceptance Testing

- **What:** Validates system against business requirements.
- **Why:** Ensures product is ready for release.
- **When:** Final stage before deployment.
- **Where:** Real-world scenarios.
- **Who:** End-users, clients.
- **How:** User acceptance tests (UAT).

Example: Client tests payroll system with real employee data.

Exercise: Create acceptance criteria for a food delivery app.

8. Regression Testing

- **What:** Ensures new changes don't break existing functionality.
- **Why:** Maintains stability after updates.
- **When:** After bug fixes, enhancements.
- **Where:** Continuous integration pipelines.
- **Who:** QA engineers, automation testers.
- **How:** Automated test suites.

Example: Re-run login tests after adding "forgot password" feature.

Exercise: Identify regression test cases after adding a discount feature in e-commerce.

9. Performance Testing

- **What:** Tests speed, scalability, and stability.
- **Why:** Ensures system meets performance benchmarks.
- **When:** Before release, during load spikes.
- **Where:** Web servers, APIs, databases.

- **Who:** Performance engineers.
- **How:** Load testing, stress testing, endurance testing.

Example: Test website response under 10,000 concurrent users.

Exercise: Design a stress test for a ticket booking system during peak hours.

10. Security Testing

- **What:** Ensures system is protected against threats.
- **Why:** Prevents vulnerabilities and breaches.
- **When:** Before deployment, periodically.
- **Where:** Web apps, APIs, databases.
- **Who:** Security testers, ethical hackers.
- **How:** Penetration testing, vulnerability scanning.

Example: Test login form against SQL injection.

Exercise: Identify possible security test cases for an online banking app.

Key Takeaway

- **Black Box** → User perspective (functional).
 - **White Box** → Developer perspective (code).
 - **Grey Box** → Hybrid perspective (integration/security).
 - **Other types** → Unit, Integration, System, Acceptance, Regression, Performance, Security.
-